QUESTION 1 – MEDIUM – DYNAMIC TABLES & SCD TYPE 1 / TYPE 2

Marks: 20

Objective

Load customer change records from an AWS S3 external stage in multiple batches and use Snowflake Dynamic Tables to maintain:

- **SCD Type 1 – latest customer values only**
- **SCD Type 2 – complete customer history**

Step 1 – Create Environment

Create:

- **Database: CUSTOMER_SCD_DB**
- **Schema: RAW**
- **CSV File Format: CSV_FMT**
- **External Stage: S3_CUSTOMER_STAGE**

Configure CSV_FMT with:

- **TYPE = CSV**
- **FIELD_DELIMITER = ','**
- **SKIP_HEADER = 1**

Configure S3_CUSTOMER_STAGE using the AWS S3 path and storage integration/credentials provided for the assessment.

Create the source table:

```
CREATE OR REPLACE TABLE CUSTOMER_SCD_DB.RAW.CUSTOMER_RAW (
    CUSTOMER_ID NUMBER,
    CUSTOMER_NAME STRING,
    EMAIL STRING,
    CITY STRING,
    LOYALTY_TIER STRING,
    UPDATED_AT TIMESTAMP_NTZ
);
```

Step 2 – Load Initial Data

Use the provided file:

customer_batch1.csv

Load the file from S3_CUSTOMER_STAGE into CUSTOMER_RAW using `COPY INTO`.

Verify:

Expected Row Count: 20

Step 3 – Create Dynamic Tables

Create both dynamic tables using:

- **TARGET_LAG = '1 minute'**
- **an available Snowflake warehouse**

A. DT_CUSTOMER_SCD1

Create:

DT_CUSTOMER_SCD1

The dynamic table must contain only the latest version of each CUSTOMER_ID based on `UPDATED_AT`.

Required columns:

- **CUSTOMER_ID**
- **CUSTOMER_NAME**
- **EMAIL**
- **CITY**
- **LOYALTY_TIER**
- **LAST_UPDATED_AT**

After the initial load:

Expected Row Count: 20

B. DT_CUSTOMER_SCD2

Create:

DT_CUSTOMER_SCD2

The dynamic table must maintain every version of each customer.

Required columns:

- **CUSTOMER_ID**
- **CUSTOMER_NAME**
- **EMAIL**
- **CITY**
- **LOYALTY_TIER**
- **EFFECTIVE_FROM**
- **EFFECTIVE_TO**
- **IS_CURRENT**
- **VERSION_NO**

Use appropriate window logic based on:

CUSTOMER_ID + UPDATED_AT

The latest record must have:

- `IS_CURRENT = TRUE`
- **EFFECTIVE_TO as NULL or an agreed future date**

After the initial load:

Expected Row Count: 20

Step 4 – Load First Change Batch

Load:

customer_batch2.csv

The file contains:

- **6 changed existing customers**
- **3 new customers**

Refresh the Dynamic Tables manually or allow the target lag to refresh them.

Verify:

- **CUSTOMER_RAW = 29 rows**
- **DT_CUSTOMER_SCD1 = 23 rows**
- **DT_CUSTOMER_SCD2 = 29 rows**

Validate:

CUSTOMER_ID = 8108

Expected behavior:

- **SCD1 → only the latest values are visible**
- **SCD2 → old and new versions are visible**
- **Old version →** `IS_CURRENT = FALSE`
- **Latest version →** `IS_CURRENT = TRUE`

Step 5 – Load Second Change Batch

Load:

customer_batch3.csv

This batch contains another LOYALTY_TIER change for:

- **CUSTOMER_ID = 8105**
- **CUSTOMER_ID = 8108**

Refresh the Dynamic Tables again.

Verify final counts:

- **CUSTOMER_RAW = 31 rows**
- **DT_CUSTOMER_SCD1 = 23 rows**
- **DT_CUSTOMER_SCD2 = 31 rows**

Display the complete SCD2 history for:

- **CUSTOMER_ID 8105**
- **CUSTOMER_ID 8108**

Each customer must show:

3 versions

Also verify that both Dynamic Tables refreshed successfully using Dynamic Table metadata or refresh history.

Submission Requirements

Submit all clear screenshots.

1. SCD Type 1 Final Output

Show DT_CUSTOMER_SCD1 after all batches are processed, including CUSTOMER_ID 8105 and 8108 with only their latest values.

2. SCD Type 2 Final History

Show DT_CUSTOMER_SCD2 for CUSTOMER_ID 8105 and 8108, clearly showing all 3 versions, EFFECTIVE_FROM, EFFECTIVE_TO, IS_CURRENT, and VERSION_NO.

3. Dynamic Table Final Validation

Show the final row counts and evidence that both Dynamic Tables refreshed successfully.

Evaluation Rubric – 20 Marks

1. Environment & Source Data – 4 Marks

The required database, schema, file format, external stage, and CUSTOMER_RAW table are configured correctly, and all supplied batches are loaded successfully.

Validation:

Verify from the final source data that CUSTOMER_RAW contains 31 rows after all three batches.

2. SCD Type 1 Dynamic Table – 5 Marks

DT_CUSTOMER_SCD1 correctly maintains only the latest row for each customer.

Validation:

Verify that the final table contains 23 unique customers and CUSTOMER_ID 8105 and 8108 show only their latest values.

3. SCD Type 2 Dynamic Table – 7 Marks

DT_CUSTOMER_SCD2 correctly maintains the full version history using effective dates, current flag, and version number.

Validation:

Verify CUSTOMER_ID 8105 and 8108 each contain 3 correctly ordered versions, with only the latest row marked as current.

4. Dynamic Table Refresh & Final Validation – 4 Marks

Both Dynamic Tables refresh successfully and produce the expected final outputs.

Validation:

Verify:

- **CUSTOMER_RAW = 31**
- **DT_CUSTOMER_SCD1 = 23**
- **DT_CUSTOMER_SCD2 = 31**

and confirm successful Dynamic Table refresh status/history.

Total: 20 Marks

Evaluation Note

Award marks primarily based on the correct final SCD Type 1 and SCD Type 2 outputs.

Intermediate setup commands do not require separate screenshots.

Partial marks may be given where a major outcome is partially working.

Missing or incorrect final evidence should not receive full marks.

QUESTION 2 – MEDIUM – SNOWPARK SQL TRANSFORMATIONS & STREAMLIT APP

Marks: 20

Objective

Load hotel booking data, perform SQL transformations using Snowpark Python, persist the transformed outputs as Snowflake tables, and build an interactive Streamlit in Snowflake application.

Step 1 – Create Environment & Load Data

Create:

- **Database: HOTEL_ANALYTICS_DB**
- **Schema: CURATED**
- **CSV File Format: CSV_FMT**
- **Internal Stage: HOTEL_STAGE**

Configure CSV_FMT with:

- **TYPE = CSV**
- **FIELD_DELIMITER = ','**
- **SKIP_HEADER = 1**

Create the source table:

```
CREATE OR REPLACE TABLE HOTEL_ANALYTICS_DB.CURATED.HOTEL_BOOKINGS (
    BOOKING_ID NUMBER,
    HOTEL_ID STRING,
    HOTEL_NAME STRING,
    CITY STRING,
    ROOM_TYPE STRING,
    BOOKING_CHANNEL STRING,
    GUESTS NUMBER,
    CHECKIN_DATE DATE,
    CHECKOUT_DATE DATE,
    ROOM_RATE NUMBER(10,2),
```

```
    DISCOUNT_PCT NUMBER,
    BOOKING_STATUS STRING
);
```

Upload the provided:

hotel_bookings.csv

to HOTEL_STAGE and load it into HOTEL_BOOKINGS using `COPY INTO`.

Verify:

Expected Row Count: 40

Step 2 – Transformation 1: Clean & Enrich

Using a Snowflake Python Worksheet / Notebook and `session.sql()`:

Create:

BOOKINGS_CURATED

Apply the following logic:

- **Exclude records where `BOOKING_STATUS = 'CANCELLED'`**
- **Add:**

NIGHTS

```
DATEDIFF(day, CHECKIN_DATE, CHECKOUT_DATE)
```

GROSS_AMOUNT

```
NIGHTS * ROOM_RATE
```

NET_REVENUE

```
GROSS_AMOUNT * (1 - DISCOUNT_PCT / 100)
```

STAY_MONTH

```
DATE_TRUNC('MONTH', CHECKIN_DATE)
```

Save the result using `save_as_table()` in overwrite mode.

Expected:

- **36 rows**
- **Total NET_REVENUE = 980,960.00**

Step 3 – Transformation 2: Top Hotel per City

Using `session.sql()` on BOOKINGS_CURATED:

Create:

TOP_HOTEL_PER_CITY

Calculate by CITY and HOTEL_NAME:

- **TOTAL_NET_REVENUE**
- **TOTAL_NIGHTS**
- **BOOKING_COUNT**

Use `RANK()` with `QUALIFY` to retain only the top hotel in each city based on TOTAL_NET_REVENUE.

Expected:

4 rows

Step 4 – Transformation 3: Monthly Revenue Trend

Using `session.sql()` on BOOKINGS_CURATED:

Create:

CITY_MONTHLY_TREND

Calculate by CITY and STAY_MONTH:

- **TOTAL_NET_REVENUE**
- **BOOKING_COUNT**

Use `LAG()` to calculate:

- **PREV_MONTH_REVENUE**
- **MOM_GROWTH_PCT**

Use:

```
(Current Revenue - Previous Revenue)
/
Previous Revenue
* 100
```

The first available month for each city must return NULL for MOM_GROWTH_PCT.

Order the output by:

- **CITY**
- **STAY_MONTH**

Expected:

17 rows

Step 5 – Build Streamlit App

Create a Streamlit in Snowflake app named:

HOTEL_REVENUE_APP

Use Snowpark `get_active_session()` and the transformed tables.

Filters

Add:

- **CITY – `st.multiselect`**
- **NIGHTS – `st.slider` from 1 to 6**

The default CITY selection must include all cities.

KPI Metrics

Display:

- **Total Net Revenue**
- **Total Bookings**
- **Average Nights**

Visuals

Display:

- **Bar chart – NET_REVENUE by ROOM_TYPE**
- **Line chart – monthly revenue trend using CITY_MONTHLY_TREND**

Additional Output

Display:

TOP_HOTEL_PER_CITY

as a table.

The CITY and NIGHTS filters must update the relevant KPIs, chart, and filtered booking data.

Submission Requirements

Submit all clear screenshots.

1. Snowpark Transformation Results

Show:

- **BOOKINGS_CURATED**
- **TOP_HOTEL_PER_CITY**
- **CITY_MONTHLY_TREND**

with evidence of the required calculated columns and final outputs.

2. Streamlit Filters & KPI Validation

Show HOTEL_REVENUE_APP with:

- **CITY multiselect**
- **NIGHTS slider**
- **Total Net Revenue**
- **Total Bookings**
- **Average Nights**

3. Streamlit Final Dashboard

Show:

- **NET_REVENUE by ROOM_TYPE bar chart**
- **monthly revenue trend line chart**
- **TOP_HOTEL_PER_CITY table**
- **filtered dashboard output**

Evaluation Rubric – 20 Marks

1. Environment & Transformation 1 – 5 Marks

The source data is loaded successfully and BOOKINGS_CURATED is created correctly.

Validation:

Verify the final curated output contains 36 rows, excludes CANCELLED bookings, includes NIGHTS, GROSS_AMOUNT, NET_REVENUE, and STAY_MONTH, and produces the expected total NET_REVENUE.

2. SQL Analytical Transformations – 6 Marks

TOP_HOTEL_PER_CITY and CITY_MONTHLY_TREND are created correctly.

Validation:

Verify:

- **top hotel is derived per city using ranking logic**
- **monthly trend uses LAG correctly**
- **MOM_GROWTH_PCT is calculated correctly**
- **expected final outputs are produced**

3. Streamlit Filters & KPI Metrics – 4 Marks

HOTEL_REVENUE_APP contains the required filters and KPI metrics.

Validation:

Verify the CITY multiselect and NIGHTS slider work correctly and the KPI values respond to the selected filters.

4. Streamlit Visuals & Final Output – 5 Marks

The dashboard displays the required charts, top-hotel table, and filtered results correctly.

Validation:

Verify the ROOM_TYPE bar chart, monthly trend line chart, TOP_HOTEL_PER_CITY table, and final filtered dashboard output.

Total: 20 Marks

Evaluation Note

Award marks primarily based on correct Snowpark SQL transformations and the final working Streamlit application.

Intermediate implementation commands do not require separate screenshots.

Partial marks may be given where a major outcome is partially working.

Missing or incorrect final evidence should not receive full marks.